

Using the MagLev Branch

Using the MagLev Branch

Generated from GemStonePharoContract load-group and steady-state API contracts.

Which Load Group To Use

- Original original/base production packages only
- Original-Tests original/base production plus the original/base tests
- MagLev the MagLev production runtime: base/original + generic core + MagLev runtime, without debugger/tool UI packages
- MagLev-Tools the interactive developer stack for GemStone workspace, GbsRemoteDebugger, inspectors, transcript, ObjectLog, and MagLev method tools
- Full the full developer stack, including MagLev-aware test packages

Load From Local Disk

```
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///absolute/path/to/GemStone-Pharo-Bridge/
src';
  load: 'MagLev'.
```

Use load: 'MagLev-Tools' if you want interactive debugging tools. Use load: 'Full' if you also want the full developer/test stack.

Load From GitHub

```
Metacello new
  baseline: 'GemStonePharo';
  repository: 'github://unicompute/GemStone-Pharo-Bridge:maglev/
src';
  load: 'MagLev'.
```

Use `load: 'MagLev-Tools'` from the same repository if you want interactive debugging tools, or `load: 'Full'` if you want the MagLev-aware test packages as well.

Clean Switching

Use a fresh image or the clean-reload path each time. Do not incrementally load `Original` and then `MagLev` into the same already-mutated image.

Common wrapper entry points:

- `make original PHARO_IMAGE="..." PHARO_WORK_DIR="..."`
- `make original-tests PHARO_IMAGE="..." PHARO_WORK_DIR="..."`
- `make full PHARO_IMAGE="..." PHARO_WORK_DIR="..." GS_USER=... GS_PASS=... GS_NETLDI_HOST=... GS_NETLDI_NAME_OR_PORT=... GEMSTONE=...`

Does The Classic Session Example Still Work?

Yes.

- `GbsSessionParameters>>login` still returns a `GbsSession`
- `GbsSession>>userGlobals` still exists
- `GbsSession>>commit` is still present as a compatibility alias for `commitTransaction`
- `GbsSession>>disconnect` still exists

This classic example still works on the MagLev branch:

```
| session dict |
session := GbsSessionParameters new
          name: 'Simple Session';
          gemStoneName: 'gs64stone';
          username: 'DataCurator';
          password: 'swordfish';
          login.

dict := Dictionary new.
dict at: 'name' put: 'Tariq'.
dict at: 'amount' put: 100.
dict at: 'currency' put: 'GBP'.

session userGlobals at: #MyTestDict put: dict.
session commit.

session disconnect.
```

A better MagLev-oriented version for new code is:

```
| session payload |
session := GbsSessionParameters new
          name: 'MagLev Session';
```

```
gemStoneName: 'gs64stone';
username: 'DataCurator';
password: 'swordfish';
"Optional but clearer for explicit routing:"
netldiHostOrIp: 'localhost';
netldiNameOrPort: '50377';
login.
```

```
[
  payload := Dictionary new
    at: 'name' put: 'Tariq';
    at: 'amount' put: 100;
    at: 'currency' put: 'GBP';
    yourself.

  session bridgeRoot at: #MyTestDict put: payload.
  session commitTransactionOrSignalConflict
] ensure: [
  session disconnect
].
```

For new code, prefer `bridgeRoot` over direct `UserGlobals` writes, and prefer `commitTransactionOrSignalConflict` or `commitTransactionWithRetryCount:` over the older `commit` alias when you want explicit transaction behavior.